Julia PIV Guide

Coastal Imaging Lab – CEOAS – OSU
Robin Shindelman

Table of Contents

Useful Paths ***1***

Process Outline ***2***

Via Linux Terminal ***3***

Via GUI ***5***

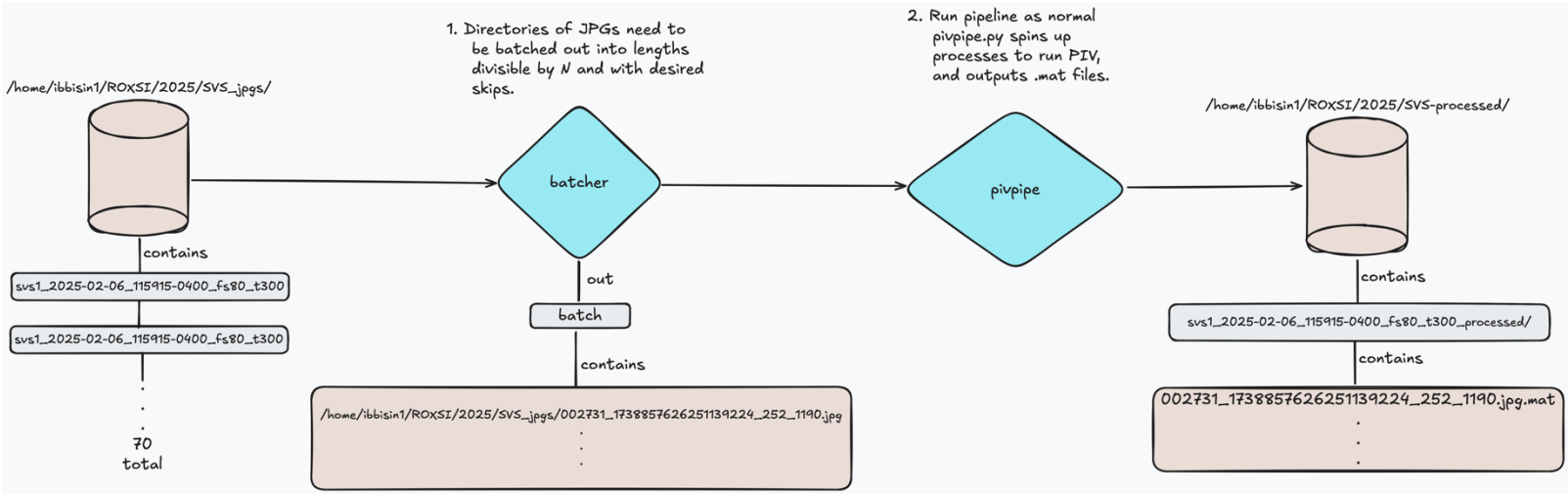
Useful Paths

pivpipe /home/server/pi/homes/shindelr/scripts/pivpipe **YAML config files** /home/server/pi/homes/shindelr/scripts/pivpipe/settings
batchscript /home/server/pi/homes/shindelr/scripts/batchscript
PIVPipelineUtility source /home/server/pi/homes/shindelr/Nearshore-PIV/PIVPipelineUtility/src/PIVPipelineUtility.jl **Executable binary for the Julia pipeline** /home/server/pi/homes/shindelr/Nearshore PIV/piv_build/bin/PIVPipelineUtility

Process Outline

There are two primary ways to run this PIV application. Through a series of Linux commands and scripts or by way of a point and click graphical user interface which is currently under development.

From a high level, the full pipeline looks like this:



batcher: a simple python script which creates batches of text files according to the number of images in the directory being prepared for processing. Each text file contains the absolute paths of every image in the batch.

It's recommend that the user do some arithmetic before this step so that you end up with a number of batches, each with a number of image paths evenly divisible by the PIV N setting. For example, an unprocessed SVS directory may contain 12 images, if N=3 for PIV, number of batches created should be set to either 2 or 4, this way each batch contains either 3 or 6 images.

Example:

```
batcher    -o  fake/path/batched_svs    -b  4    fake/path/original_svs
```

Installation:

```
python3 -m pip install --user
/home/server/pi/homes/shindelr/scripts/batchscript
```

pivpipe: a python script which uses multiprocessing to spawn many Julia processes, each tasked with running PIV on a single batch text file. The pivpipe script has many flags which can be used to specify the exact settings desired for PIV.

While a user can choose these settings from the command line, the use of a YAML configuration file is much easier. A nice reference page can be found here. Use the "config" option to specify a path to the YAML file.

Example:

```
pivpipe    -c  settings/test_run.yaml
```

Installation:

```
python3 -m pip install --user /home/server/pi/homes/shindelr/scripts/pivpipe
```

PIVPipelineUtility: a Julia program that handles the I/O of each image being processed by PIV. This program is compiled into a binary and executed by the pivpipe script. This file can be modified in src/PIVPipelineUtility/ which can be found via the path shown in useful paths.

Recompile from the command line using the steps below.

```
1. cd /home/server/pi/homes/shindelr/NearshorePIV/PIVPipelineUtility
2. ml eb-sw Julia
3. julia --project=.
4. using PackageCompiler
5. create_app("/home/server/pi/homes/shindelr/Nearshore-
   PIV/PIVPipelineUtility", "../piv_build";
   precompile_execution_file="precompile.jl", force=true)
```

Step 5 can sometimes take upwards of 10 minutes. The compiled binary will be found in /home/server/pi/homes/shindelr/Nearshore PIV/piv_build/bin

Via Linux Terminal

A recommended structure for processing the 2025 data may look something like the one below. SVS_jpgs currently has 70 SVS collection directories, each containing thousands of images.

```
2025/
├── postProcessing
│   ├── piv_batches
│   ├── piv_mat_out
│   └── settings
└── SVS_jpgs
    └── svs1_2025-02-06_124837-0400_fs80_t300
```

1. Decide on configuration settings for processing. The available options are: N, crop factors, final window size, overlap, input directory, output directory, quiet, down sample factor, number of processes, and save cropped/down sampled images.
2. Navigate to the settings directory: `cd 2025/postProcessing/settings/`
3. Build configuration YAML: `touch svs1_2025-02-06_124837-0400_fs80_t300.yaml`
4. Open `svs1_2025-02-06_124837-0400_fs80_t300.yaml` with the user's preferred text editor and enter the configuration settings. A YAML settings file might look like the one below.

```
# config.yaml
N: 2

crop_factor: "1, 2448, 1, 2048"  # Uncropped

final_win_size: 16

ol: 0.5

# Absolute paths
input: "/home/ibbisin1/ROXSI/2025/piv_batches/svs1_2025-02-06_124837-
0400_fs80_t300-in"

output: "/home/ibbisin1/ROXSI/2025/piv_mat_out/svs1_2025-02-06_124837-
0400_fs80_t300-out"

quiet: false

downsample_factor: 1.0

NPROC: 40

save_images: false
```

5. Create the input and output directories:

```
mkdir piv_batches/svs1_2025-02-06_124837-0400_fs80_t300-in
mkdir piv_mat_out/svs1_2025-02-06_124837-0400_fs80_t300-out
```

6. Using the batcher script described in the process outline, create batches of text files for multi-processing. In this case, there are 24,000 images in `svs1_2025-02-06_124837-0400_fs80_t300-in` and the YAML configuration N was set to 2 and NPROC was set to 40. With 40 batches, each text file should have 600 image paths, perfectly divisible by N=2. Create the batches, modifying paths as needed:

```
batcher -o piv_batches/svs1_2025-02-06_124837-0400_fs80_t300-in/ -b 40
../SVS_jpgs/svs1_2025-02-06_124837-0400_fs80_t300
```

7. Verify that each text file contains 600 absolute image paths, a method of verifying this may look something like this bash command. There should be 40 lines of 600 printed to the console:

```
for f in piv_batches/svs1_2025-02-06_124837-0400_fs80_t300-in/*.txt;
do wc -l < "$f";
done
```

8. At this point, the directory structure should appear as:

```
2025/
├── postProcessing
│   ├── piv_batches
│   │   └── svs1_2025-02-06_124837-0400_fs80_t300-in
│   │       ├── batch1
│   │       ├── batch2
│   │       ├── batch3
│   │       .
│   │       .
│   │       └── batch40
│   ├── piv_mat_out
│   │   └── svs1_2025-02-06_124837-0400_fs80_t300-out
│   └── settings
│       └── svs1_2025-02-06_124837-0400_fs80_t300.yaml
└── SVS_jpgs
    └── svs1_2025-02-06_124837-0400_fs80_t300
```

9. Before running PIV for the first time in a while, run the 1-pair precompile PIV test. Julia releases packages and libraries to the OS garbage collector over time to keep memory overhead down for unused compile programs. Running PIV on a quick one pair test will force the program to compile packages now, rather than on x-number of processes when the real PIV happens.

```
pivpipe -c /home/server/pi/homes/shindelr/scripts/pivpipe/settings/1-pair-precompile.yaml
```

10. Finally – run PIV! This step can take quite a long time.

```
pivpipe -c settings/svs1_2025-02-06_124837-0400_fs80_t300.yaml
```

11. Repeat, creating new directories for each of the svs* collection directories. Different YAML files may be needed to modulate the number of processes being used by the computer depending on the number of images in each directory and their batch numbers.

Via GUI

TODO